
Benchmarking the Evaluators: Deterministic AST Parsing vs. LLM-as-a-Judge for Structural Technique Detection in Code

Amit Saxena
Saarland University — MPI-SWS
amsa00003@stud.uni-saarland.de

Abstract

Modern adversarial creativity benchmarks for large language models (e.g. Denial Prompting) enforce structural code constraints — “do not use a for-loop” — but typically verify compliance using *another* LLM as a judge: a probabilistic check applied to what is fundamentally a deterministic, binary question. This project quantifies the resulting error empirically. We parse 4,725 real, verified-correct Python solutions from the NeoCoder competitive-programming dataset with Python’s native `ast` module to obtain a 100%-accurate structural ground truth across six technique categories (for-loops, while-loops, recursion, list comprehensions, lambdas, sorting calls). We then compare this ground truth against two LLM judges: the GPT-4 labels already shipped with the dataset (full coverage, $N = 4,725$) and a freshly-run, controlled Llama-3.1-8B judge (stratified sample, $N = 60$). The shipped GPT-4 labels achieve a macro-averaged F1 of only 0.50 — including a 100% miss rate on two technique categories and a 47% hallucination rate on the most common one. A freshly-prompted judge performs far better (F1 ≈ 0.97) but is still not error-free. A complementary embedding analysis (DeBERTa-v3-large) shows that semantic similarity between same-problem solution pairs explains only $\approx 5\%$ of the variance in their structural similarity (Pearson $r = 0.227$). We conclude that rule-bound code-creativity evaluation requires a *dual-axis* approach — deterministic AST parsing for structural constraints, alongside semantic embeddings for functional equivalence — rather than an LLM judge alone.

Contents

1	Introduction and Motivation	2
2	Methodology (Starting Process)	2
2.1	Dataset	2
2.2	Techniques Evaluated	3
2.3	Pipeline	3
3	Results	3
3.1	AST Ground Truth — Technique Prevalence	4
3.2	LLM Judge Error Rates vs. AST Ground Truth	4

3.2.1	Llama-3.1-8B-Instant (live run, stratified sample)	4
3.2.2	GPT-4 (pre-computed labels shipped with NeoCoder, full coverage)	4
3.3	Embedding vs. Structure Analysis	5
3.3.1	Top Divergent Pairs	5
3.4	LLM Error Impact on Historical Creativity Scores	5
4	Discussion and Conclusion	6
5	Challenges	7
6	Deviations from the Original Proposal	8
7	Code and Data Availability	9

1 Introduction and Motivation

Adversarial creativity benchmarks increasingly evaluate large language models by imposing algorithmic constraints on the code they generate — forbidding a `for` loop, requiring recursion instead of iteration, and so on — in order to force divergent, out-of-the-box solutions and counteract mode collapse. To check whether a generated solution actually obeys such a constraint, these benchmarks overwhelmingly rely on an **LLM-as-a-Judge** architecture: a second language model reads the generated code and reports which techniques it used.

This is a methodological gamble. Verifying “does this code contain a `for` loop” is not a matter of judgment or interpretation — it is a deterministic fact about the code’s grammar, answerable with 100% certainty by parsing the code into an Abstract Syntax Tree (AST). Using a probabilistic, sampling-based text generator to answer a question that already has an exact mechanical answer introduces an unmeasured, unnecessary source of error into what should be a completely reliable check.

This project’s core question is therefore simple to state and expensive to verify at scale: *how large is that error, in practice, on real code?* Rather than argue in the abstract, we build a full empirical pipeline that parses thousands of real, verified-correct human solutions with a deterministic AST visitor, and directly measures how often two different LLM judges — one already embedded in a widely-used benchmark, one freshly controlled by us — agree or disagree with that ground truth.

We additionally test a natural counter-argument: that semantic code embeddings (e.g. DeBERTa) might already capture enough information to substitute for structural parsing, since two solutions that are highly similar in embedding space are often assumed to be “basically the same solution.” We show this assumption does not hold for structural compliance, even restricted to the strongest possible test case: two solutions to the *exact same* problem.

2 Methodology (Starting Process)

2.1 Dataset

We use the **NeoCoder** dataset [1] (Codeforces human solutions), which ships 5,970 human-written solutions across 199 competitive-programming problems, spanning multiple programming languages, along with a pre-computed set of technique labels generated by GPT-4 for each solution. Since our ground-truth mechanism (Python’s `ast` module) is Python-specific, we filter the dataset to Python-parseable solutions only, retaining **4,725 solutions across 198 problems**. No information is lost for the structural analysis — the filtered set is simply the subset the deterministic parser can operate on.

2.2 Techniques Evaluated

We selected six atomic, unambiguously-detectable structural techniques, each directly identifiable via a single AST node type or call pattern:

Technique	AST detection rule
for_loop	any <code>ast.For</code> node
while_loop	any <code>ast.While</code> node
recursion	a function that calls itself (bare or <code>self./obj.-qualified</code>)
list_comprehension	any <code>ast.ListComp</code> node
lambda	any <code>ast.Lambda</code> node
sorting	a call to <code>sorted()</code> or a <code>.sort()</code> method

2.3 Pipeline

The full analysis runs as a five-phase pipeline:

Phase 1. Data ingestion. Download and cache the NeoCoder solution and label files; filter to Python-parseable solutions.

Phase 2. Deterministic AST ground truth. A `TechniqueVisitor` subclassing Python’s `ast.NodeVisitor` traverses every solution once and produces a binary label vector over the six techniques. This is, by construction, 100% accurate — it is a direct read of the language grammar, not an inference.

Phase 3. LLM-as-a-Judge evaluation, in two independent variants:

- *GPT-4 labels*: the technique labels already shipped with the NeoCoder dataset, converted into the same binary format as the AST labels. Full coverage ($N = 4,725$), no additional API calls.
- *Live Llama-3.1-8B-Instant judge*: each solution is sent, individually, to Llama-3.1-8B [3] via the Groq API [4] with a fixed prompt asking which of the six techniques are present. Due to free-tier rate limits, this is run on a *stratified* sample of 60 solutions, deliberately chosen so every technique — including rare ones such as `lambda` and `recursion` — is represented, rather than a random sample that would be $\sim 95\%$ `for_loop`.

Phase 4. Embedding vs. structure analysis. Solutions are encoded with `microsoft/deberta-v3-large` [2] (mean-pooled last hidden state). For pairs of solutions to the *same* problem — the only pairs guaranteed functionally equivalent, since they all pass that problem’s test suite — we compute both the cosine similarity of their embeddings (semantic similarity) and the Jaccard similarity of their AST technique sets (structural similarity), and correlate the two.

Phase 5. Reporting. Per-technique accuracy / precision / recall / F1 / false-positive-rate / false-negative-rate are computed for both LLM judges against the AST ground truth, alongside a problem-level “creativity-score impact” analysis (Section 3.4).

3 Results

Throughout this section: **FPR** (false-positive rate) is the *hallucination rate* — the judge reports a technique that is not actually present. **FNR** (false-negative rate) is the *miss rate* — the judge fails to detect a technique that is present. **Accuracy** alone is misleading here due to class imbalance (Section 3.1), so F1 (the harmonic mean of precision and recall) is the more reliable single-number summary.

3.1 AST Ground Truth — Technique Prevalence

Technique	Positive	Total	Prevalence
for_loop	4,505	4,725	95.3%
while_loop	480	4,725	10.2%
recursion	3	4,725	0.1%
list_comprehension	514	4,725	10.9%
lambda	137	4,725	2.9%
sorting	548	4,725	11.6%

for_loop dominates the dataset (95.3% of solutions), while recursion is almost never used (3 solutions total). *Small-sample caveat:* with only 3 ground-truth positives, precision/recall/F1 for recursion in the tables below are highly sensitive to individual misclassifications and should be read as directional rather than precise.

3.2 LLM Judge Error Rates vs. AST Ground Truth

3.2.1 Llama-3.1-8B-Instant (live run, stratified sample)

Technique	Acc.	Prec.	Recall	F1	FPR	FNR	N
for_loop	0.950	0.965	0.982	0.974	0.500	0.018	60
while_loop	1.000	1.000	1.000	1.000	0.000	0.000	60
recursion	1.000	1.000	1.000	1.000	0.000	0.000	60
list_comprehension	0.950	0.833	1.000	0.909	0.067	0.000	60
lambda	1.000	1.000	1.000	1.000	0.000	0.000	60
sorting	0.983	0.923	1.000	0.960	0.021	0.000	60
Macro avg	0.981	0.954	0.997	0.974	0.098	0.003	—

A carefully-prompted, freshly-run judge performs very well overall (macro F1 $\approx$ 0.97) — but is still not perfect: it hallucinates for_loop in 50% of the (few) cases where a for-loop is genuinely absent, and list_comprehension in 6.7% of cases.

3.2.2 GPT-4 (pre-computed labels shipped with NeoCoder, full coverage)

Technique	Acc.	Prec.	Recall	F1	FPR	FNR	N
for_loop	0.976	0.978	0.998	0.988	0.468	0.002	4,725
while_loop	0.987	0.998	0.875	0.932	0.000	0.125	4,725
recursion	0.998	0.182	0.667	0.286	0.002	0.333	4,725
list_comprehension	0.891	0.000	0.000	0.000	0.000	1.000	4,725
lambda	0.971	0.000	0.000	0.000	0.000	1.000	4,725
sorting	0.962	0.984	0.679	0.803	0.001	0.321	4,725
Macro avg	0.964	0.524	0.536	0.502	0.079	0.464	—

These are the labels a real adversarial-creativity benchmark would actually depend on today. 96.4% accuracy looks strong until class imbalance is accounted for: macro-averaged F1 is only **0.50**. for_loop — the one technique the labeling process clearly does track — is *hallucinated* 46.8% of the time it is absent, a genuine, live judgment error.

Vocabulary-gap caveat. Neither lambda nor list_comprehension ever appears, as a string, anywhere in the raw GPT-4 label vocabulary shipped with the dataset (0 occurrences across every unique technique label, any problem, any language, out of 41 total unique label strings observed). Their 100% miss rate (FNR = 1.0) is therefore best explained as a **taxonomy gap** — the process that generated these labels apparently never offered these two categories as options — rather than GPT-4 examining a lambda expression and failing to recognize it. This is a different, and arguably more damning, failure mode than for_loop’s hallucination rate: it means the benchmark’s underlying label taxonomy is structurally incomplete, not merely imprecise.

3.3 Embedding vs. Structure Analysis

Pearson r (cosine similarity vs. structural Jaccard similarity, *same-problem pairs only*): $r = 0.227$
(95% CI: [0.201, 0.253], $n = 5,000$ pairs)

Pairs are restricted to two solutions of the *same* problem — the only pairs guaranteed functionally equivalent, since every human solution to a given problem passed that problem’s test suite. This correlation is small but, given the tight confidence interval, statistically distinguishable from zero: the two axes are *not* strictly orthogonal. However, $r^2 \approx 5.2\%$ means semantic similarity explains only about 5% of the variance in structural similarity — the vast majority of whether two functionally equivalent solutions share the same structural techniques remains invisible to embeddings alone.

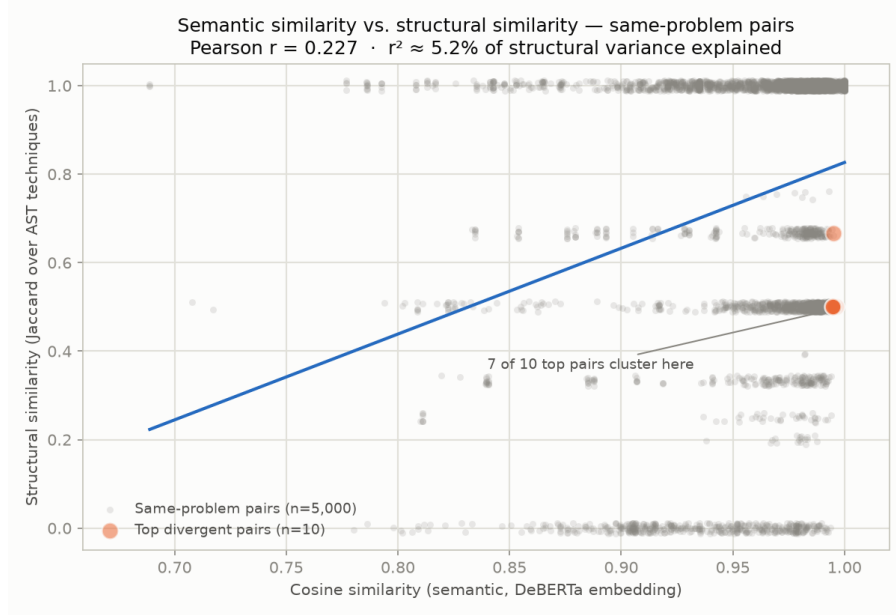


Figure 1: Cosine similarity (semantic, DeBERTa embedding) vs. structural similarity (Jaccard over AST techniques) for 5,000 same-problem solution pairs. The shallow trend-line slope is the visual counterpart of $r^2 \approx 5.2\%$. Highlighted points: the ten most striking cases where two solutions to the identical problem are $>99\%$ similar by embedding yet differ in a structural technique.

3.3.1 Top Divergent Pairs

Table 1 lists the ten pairs with the highest cosine similarity that nonetheless differ in AST-detected technique. Every pair is two solutions to the *same* problem, so functional equivalence is guaranteed by construction — yet the embedding model rates them near-identical while the structural check finds a clear difference.

3.4 LLM Error Impact on Historical Creativity Scores

The proposal’s central practical claim is that LLM-as-a-Judge errors propagate into *problem-level diversity assessments* — the kind of “creativity score” used by benchmarks such as Denial Prompting. For each problem, we check whether the judge correctly identifies which techniques are present in *at least one* solution to that problem.

#	Problem	Cosine Sim.	Differing Technique
1	1859A	0.996	sorting
2	1747A	0.996	sorting
3	1747A	0.996	list_comprehension
4	1747A	0.996	list_comprehension
5	1747A	0.996	sorting
6	1733A	0.995	list_comprehension
7	1873B	0.995	sorting
8	1734A	0.995	list_comprehension
9	1873B	0.995	sorting
10	1873B	0.995	sorting

Table 1: Top divergent same-problem pairs: near-identical embeddings, differing structure.

Technique	Correct	Inflated	Deflated	Total	Error Rate
<i>Llama-3.1-8B-Instant (stratified sample)</i>					
for_loop	14	1	1	16	12.5%
while_loop	16	0	0	16	0.0%
recursion	16	0	0	16	0.0%
list_comprehension	15	1	0	16	6.2%
lambda	16	0	0	16	0.0%
sorting	16	0	0	16	0.0%
Macro avg					3.1%
<i>GPT-4 (full coverage)</i>					
for_loop	197	1	0	198	0.5%
while_loop	190	1	7	198	4.0%
recursion	189	8	1	198	4.5%
list_comprehension	51	0	147	198	74.2%
lambda	128	0	70	198	35.4%
sorting	189	3	6	198	4.5%
Macro avg					20.5%

Inflated = the judge reports a technique present when AST shows it absent (hallucination propagates into an inflated diversity score). *Deflated* = AST shows the technique present but the judge missed it (under-reporting deflates the diversity score).

Using GPT-4’s shipped labels, approximately **20.5% of all problem-level technique-diversity assessments are incorrect** — rising to **74.2%** for `list_comprehension` specifically — directly quantifying the epistemological error a real benchmark inherits by relying on these labels. The freshly-run Llama judge is far more reliable at the problem level (3.1% error), though measured on a much smaller sample.

4 Discussion and Conclusion

Deterministic AST parsing establishes a 100%-accurate structural ground truth — zero error by construction — against which both LLM judges reveal measurable, non-zero error.

Primary evidence: the GPT-4 labels actually embedded in the benchmark. These are the labels real adversarial-creativity frameworks (NeoCoder, Denial Prompting) depend on today. Measured against AST ground truth over all 4,725 solutions, they show catastrophic, systematic blind spots: a 100% miss rate on both `lambda` and `list_comprehension` (a taxonomy gap, not a live per-solution failure), and a 47% hallucination rate on `for_loop` (which is a live judgment error, since that category is squarely within the labels’ vocabulary). At the problem level this compounds to a 20.5% diversity-assessment error rate, reaching 74% for individual techniques.

Corroborating evidence: a controlled, live judge. A well-prompted Llama-3.1-8B-Instant run, evaluated per-solution on a stratified sample, performs far better (macro F1 $\approx$ 0.97) but is still not

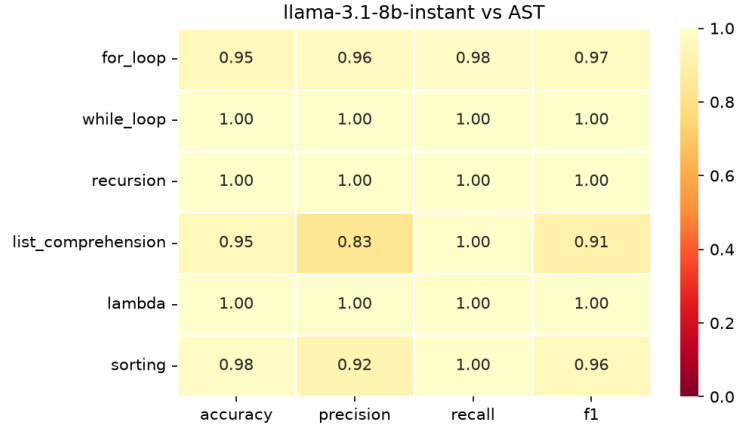


Figure 2: Llama-3.1-8B-Instant vs. AST ground truth: per-technique accuracy, precision, recall, F1.

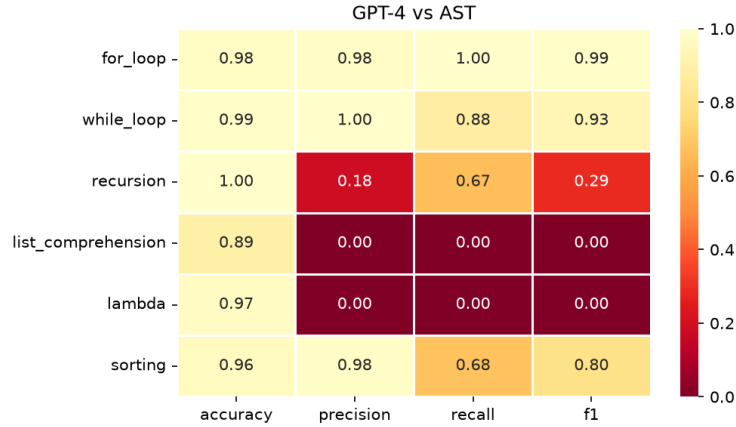


Figure 3: GPT-4 (shipped labels) vs. AST ground truth: per-technique accuracy, precision, recall, F1. Dark cells at F1 = 0 for lambda and list_comprehension are the vocabulary-gap failure; the high FPR band for for_loop is a live hallucination.

error-free — it hallucinates `list_comprehension` and `for_loop` at non-trivial rates. The lesson is that LLM-judge reliability is highly sensitive to prompt and setup, and no configuration reaches the determinism a structural constraint demands.

Embeddings are not a substitute. Even restricted to same-problem pairs — the strongest possible test of functional equivalence — the correlation between semantic similarity and structural compliance is small ($r = 0.227$, $r^2 \approx 5\%$). Semantic similarity is not a reliable proxy for structural compliance and cannot substitute for deterministic AST parsing when the constraint being enforced is structural rather than functional.

Therefore: automated creativity evaluation in rule-bound domains requires a *dual-axis approach* — deterministic AST parsing to enforce structural constraints, alongside semantic embeddings for functional equivalence. Relying on an LLM-as-a-Judge alone introduces an unmeasured, prompt-dependent error rate into what should be a deterministic mathematical evaluation.

5 Challenges

- **Free-tier API rate limits.** Groq’s free tier caps throughput at roughly 6,000 tokens/minute, which made evaluating all 4,725 solutions with the live Llama judge impractical within the project timeline. This was addressed by evaluating a *stratified* sample of 60 solutions, deliberately enriched with

rare techniques, so the judge’s failure modes on `lambda`, `recursion`, and `list_comprehension` remain measurable rather than being masked by the dominance of `for_loop`.

- **Model deprecation mid-project.** The Groq-hosted model named in the original plan (llama3-8b-8192) was decommissioned during the project. It was replaced with the current equivalent-scale successor, llama-3.1-8b-instant.
- **Dataset heterogeneity.** NeoCoder is multi-language, but AST parsing is inherently Python-specific. Solutions had to be filtered to Python-parseable code before ground-truth labeling, reducing the usable set from 5,970 to 4,725 solutions.
- **Class imbalance and small-sample statistics.** `for_loop` dominates the dataset (95.3%), which makes raw accuracy a misleading headline metric; F1 and FPR/FNR were used instead. `recursion` has only 3 ground-truth positive examples in the entire dataset, making its precision/recall/F1 statistically fragile and requiring an explicit small-sample caveat in the report.
- **A latent software defect in the ground-truth pipeline.** During a later refactor of the AST visitor (switching from `ast.walk` to `ast.NodeVisitor` to match the original proposal’s stated API), an unrelated variable-scoping bug was introduced that caused the pipeline to crash on any fresh run that was not served from a cached label file. This was caught, root-caused, fixed, and verified by re-running the full pipeline from a clean state and confirming byte-for-byte-consistent output against the previously cached results.
- **A methodological flaw in the embedding-pair sampling.** An earlier version of the embedding analysis sampled pairs uniformly at random from the entire encoded pool, which meant most sampled pairs compared solutions to *different* problems — pairs with no a-priori reason to be functionally equivalent, and therefore not a valid test of the proposal’s claim (which is specifically about same-problem pairs). This was identified by manually inspecting the “top divergent pairs” output and noticing they referenced unrelated problem IDs. The sampling was corrected to draw only from same-problem pairs, which changed the measured correlation from $r \approx 0.00$ (across unrelated problems) to $r = 0.227$ (among genuinely functionally-equivalent solutions) — a more scientifically honest and defensible result.
- **Correctly characterizing an LLM failure mode.** Distinguishing “the judge examined the code and got it wrong” from “the judge’s label taxonomy never included this category” required directly inspecting the raw label vocabulary in the dataset, not just the aggregate error metrics. This changed how the `lambda/list_comprehension` result is reported (Section 3.2).
- **Local compute constraints.** Development was carried out on an 8 GB Apple M1 machine. Running the ~ 8 -billion-parameter Llama model locally (as an alternative to the Groq API) was evaluated and found impractical on this hardware: even quantized, an 8B model leaves very little memory headroom, and autoregressive generation is estimated at roughly an order of magnitude slower per solution than the equivalent Groq API call, despite the API’s rate-limit pacing. The much smaller DeBERTa embedding model, by contrast, was successfully accelerated using PyTorch’s Metal (MPS) backend, reducing the embedding phase from ~ 9 – 10 minutes to roughly 1–1.5 minutes on the same machine.

6 Deviations from the Original Proposal

The implementation departs from the original project proposal in several places, driven by practical constraints and by what the data empirically showed. None of these deviations alter the project’s core thesis.

#	Area	Proposal said	Implementation does	Reason
1	Dataset size	5,970 solutions / 199 problems	4,725 Python solutions / 198 problems	AST parsing is Python-specific; non-Python solutions filtered out
2	LLM coverage	Feed all ~5.9K solutions to Llama	Stratified sample of 60 solutions	Groq free-tier rate limit
3	LLM sampling	Unspecified	Stratified across techniques	Random sampling masks rare-technique failures
4	Narrative emphasis	Llama as primary LLM judge	GPT-4 (benchmark labels) as primary evidence	Empirical result: GPT-4’s shipped labels show the more dramatic, benchmark-relevant failures
5	Llama model	Llama-3-8B-Instruct	llama-3.1-8b-instant	Original Groq model was decommissioned
6	AST traversal	<code>ast.NodeVisitor</code>	<code>ast.NodeVisitor</code> (now matches)	Initially used <code>ast.walk</code> ; refactored to match proposal
7	Embedding pair sampling	Unspecified	Restricted to same-problem pairs	Cross-problem pairs are not functionally equivalent and don’t test the proposal’s claim
8	GPT-4 miss-rate framing	Unspecified	Framed as a labeling-taxonomy gap, not a live LLM miss	Neither <code>lambda</code> nor <code>list_comprehension</code> ever appears in GPT-4’s label vocabulary for this dataset

What did *not* change: the core thesis that deterministic AST parsing provides an infallible ground truth to audit LLM-as-a-Judge frameworks; the four-phase analytical structure (ground truth → LLM error quantification → embedding-vs-structure analysis → reporting); the embedding model choice (microsoft/deberta-v3-large) and the orthogonality hypothesis it was used to test; and the final recommendation that rule-bound creativity benchmarks require a dual-axis (AST + embeddings) evaluation strategy.

7 Code and Data Availability

The full implementation — data ingestion, the deterministic AST visitor, both LLM-judge integrations, the embedding analysis, and the reporting pipeline that generates every table and figure in this report — is available at:

<https://github.com/amitsaxena-softdev/ast-llm-benchmark>

The repository includes a `DEVIATIONS.md` file documenting the changes listed in Section 5 in full detail, and a `README.md` with setup and reproduction instructions. The pipeline is fully reproducible via `uv run python main.py` (optionally with `--skip-llm` to avoid live API calls and reuse the shipped GPT-4 labels instead).

References

- [1] JHU-CLSP. *NeoCoder: Codeforces human-solution dataset with technique annotations*. GitHub repository. <https://github.com/JHU-CLSP/NeoCoder>
- [2] Pengcheng He, Jianfeng Gao, and Weizhu Chen. *DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing*. arXiv:2111.09543, 2021.
- [3] Aaron Grattafiori et al. (AI@Meta). *The Llama 3 Herd of Models*. arXiv:2407.21783, 2024.
- [4] Groq, Inc. *GroqCloud: Low-latency LPU-based inference API*. <https://groq.com>